

- Lab 02: Scanning Mastering Nmap
- Nom de l'enseignant : Pr Anass Sebbar
- Niveau : 4th Year Cybersecurity Track

Lab n° 02

Scanning Phase – Mastering Nmap

1- Objective :

The purpose of this lab is to explore advanced network scanning techniques by Mastering Nmap Scan, including different techniques like NSE scripting engine, automation, firewall/IDS evasion techniques and packet fragmentation. In order to understand well the concept functioning of scanning process for each attack, you will capture and analyse traffic using Wireshark or tshark

Lab Outline :

- Part 1: introduction to Nmap
- Part 2: Mastering Nmap
- Part 3: OS Fingerprinting
- Part4: Firewall and IDS/IPS Evasion

Instructions:

The lab report must be submitted one week after the lab session on the Moodle application by the deadline mentioned on the platform.

The lab work must be done individually in class, but the report is to be submitted in groups of up to 2 students. Lab groups must remain the same for all reports throughout the semester.

ATTENTION:

This work demonstrates some basic techniques that can be used to commit illegal acts. This is considered useful within the course to better understand the types of attacks that can be carried out on systems we aim to protect, BUT these tests should not be applied to real systems without written authorization.

Network Mapper Scanner:

1. Introduction to Nmap:

Nmap (Network Mapper) is an open-source utility used for both network exploration and security auditing. Nmap allows us to:

- Identify hosts present on the network.
- Detect the services running on those hosts and their versions.
- Determine which packets and ports are filtered by firewalls.
- Detect the operating systems of machines connected to the network, along with their versions.

The objective of this part of the lab is to effectively learn how to use this powerful and versatile tool. Keep in mind that, while Nmap is a valuable tool for security, it can also be used for malicious purposes, so it must be used responsibly.

In this lab, we will be performing a variety of scans using Nmap from a Kali Linux virtual machine, targeting two other virtual machines running Windows 7 and Ubuntu. We will also use Wireshark to capture and analyze traffic during each scan.

2. Nmap Network Scanning

Test 1: Host Discovery on the Network

We will begin by scanning the network to identify active devices. For this, we will use the ping scan (-sP option).

1. Start traffic capture in Wireshark.
2. Run the following command to scan the IP address range from 192.168.1.1 to 192.168.1.10:

```
# nmap -sP 192.168.1.1-10
```

3. Analyze the output from Nmap and the traffic captured by Wireshark. What protocol does Nmap use with the -sP option to detect devices on the network?
4. Use the -vv option to enable verbose mode for more detailed output on the test:

```
# nmap -sP -vv 192.168.1.1-10
```

5. Use the -sn option to perform a host discovery scan on the entire 192.168.1.0/24 subnet

```
# nmap -sn -vv 192.168.1.0/24
```

- How many hosts responded to the ICMP requests?
- Did Wireshark capture any other protocol responses apart from ICMP?

Test 2: TCP SYN Scan

1. Briefly explain how the TCP SYN scan works, including its advantages and disadvantages.
2. On the Linux machine, start Netcat to listen on ports 21, 53, 110, and 25:

```
# nc -l -p 21 & nc -l -p 53 & nc -l -p 110 & nc -l -p 25
```

3. Start Wireshark to capture traffic.

4. From Kali Linux, run a TCP SYN scan on a target machine (such as a metasploitable or Windows 7 system) using the following command (adjust the IP to match the target machine):

```
# nmap -sS 192.168.1.1
```

Note: For Windows tests, ensure the firewall is disabled.

- Use the filter `tcp.flags.syn==1 && tcp.flags.ack==0` to view SYN packets sent to the target.
- Analyze the SYN/ACK and RST responses to identify open and closed ports.

5. After identifying open and closed ports on the target machine, run a TCP SYN scan on a specific open port:

```
# nmap -sS 192.168.1.1 -p <port_number>
```

or alternatively, you can scan all ports from 1 to 65535:

```
# nmap -sS -p 1-65535 192.168.1.1
```

Analyze the messages captured by Wireshark when scanning an open port.

6. Detect the version of services running on open ports

```
# nmap -sV 192.168.1.10
```

- Capture the service responses by filtering TCP packets on the ports that were scanned.
- Use `tcp.port==[port_number]` to filter traffic on specific ports

7. Now, scan a closed port:

```
# nmap -sS 192.168.1.1 -p <port_number>
```

Compare the Wireshark captures for the open and closed ports.

3. OS Fingerprinting

Nmap also allows you to identify the **operating system** (OS) of the scanned machines using the **OS fingerprinting** feature (-O option).

1. Perform an OS fingerprinting scan with verbose mode:

```
# nmap -O 192.168.1.x
```

```
# nmap -O -vv 192.168.1.x
```

2. Perform these tests on the virtual machines you are working with, including **Windows 7**, **ubuntu**, and **metasploitable** (or other systems).
3. Test with the personal firewall both **enabled** and **disabled** on the **Windows** machines. What differences do you notice?

4. Compare the OS fingerprints obtained for different operating systems to understand how each OS implements the **TCP/IP stack** differently.

Test: Aggressive Scanning

1. **Objective:** Perform aggressive scanning to gather as much information as possible in a short time.

nmap -A 192.168.1.10

2. **Wireshark Analysis:**

- Capture the different types of packets sent by Nmap. Observe how it combines service detection, OS detection, and traceroute in the aggressive scan.
- Filter for ICMP, TCP, and UDP packets for comprehensive analysis.

3. **Questions:**

- What additional information was gathered by the aggressive scan?
- How did different packet types (ICMP, TCP, UDP) appear in Wireshark?

4. Firewall and IDS/IPS Evasion

The goal of this part is to perform packet fragmentation to evade firewalls or Intrusion Detection Systems (IDS) and analyze the resulting network traffic using Wireshark. You will use a series of Nmap commands to execute the scan while observing how fragmented packets behave.

1. Start **Wireshark** and capture traffic while performing each scan.
2. Use the following commands and analyse the results.
3. Use the following filter **ip.frag_offset > 0** to analyze fragmented packets.
4. Observe how fragmented packets are reassembled by the target.

Command	Purpose	Options Explanation
nmap -f 192.168.1.10	Perform a basic packet fragmentation scan on the target.	-f: Breaks the packets into smaller fragments to evade detection by firewalls or IDS.
nmap -sS -sV -F -D 10.0.0.1,10.0.0.3 target	Perform a SYN scan with service version detection and decoys to obscure the scan's origin.	-sS: SYN scan (stealth scan). -sV: Service version detection. -F: Fast scan (scans the most common 100 ports). -D: Use decoy IPs to hide the real source of the scan. In this case, 10.0.0.1 and 10.0.0.3 are decoy IPs.

nmap -sS -sV -F -D RND:3 nmap.scanme.org	Perform a SYN scan with random decoys.	-D RND:3: Randomly selects 3 decoy IP addresses to mask the source of the scan. The target sees multiple IPs instead of just the real source.
nmap -sS -sV -F -f -D 171.124.180.173 nmap.scanme.org	Perform a SYN scan with service version detection, packet fragmentation, and a decoy.	-f: Fragment the packets to evade detection.
nmap -sS -sV -F -f --send-eth -D 171.124.180.173 nmap.scanme.org	Perform a SYN scan with packet fragmentation and raw Ethernet frame transmission.	--send-eth: Send packets at the Ethernet (link layer) level instead of relying on the OS network stack. This can evade some defenses.
nmap -sS -sV -F -f -mtu 16 --send-eth -D 171.124.180.173 nmap.scanme.org	Perform a SYN scan with extreme fragmentation by setting the MTU to 16 bytes.	-mtu 16: Set the Maximum Transmission Unit (MTU) to 16 bytes, creating even smaller packet fragments. --send-eth: Send packets at the Ethernet level.

Questions

1. Did the fragmented packets bypass the firewall or IDS?

Analyze whether the fragmentation helped evade detection by any active security mechanisms like firewalls or IDS.

2. How were the fragmented packets handled and reassembled by the target, as seen in Wireshark?

Look for signs of how the target system handled and reassembled the fragmented packets. Were there any delays or issues during reassembly?